



Evaluation of Algorithms

Saurab Neupane
CSCE 3301-202

Dr. Mahmud Hasanul

1. Sorting Algorithms

1.1 Bubble Sort

Bubble Sort repeatedly scans the array and swaps adjacent elements that are out of order. Each full pass moves the largest unsorted element to its correct position. While simple to implement, it is inefficient on large inputs.

Time Complexity: $O(n^2)$ worst/average, $O(n)$ best (already sorted).

Space Complexity: $O(1)$.

Code:

```
void bubbleSort(vector<int>& arr) {  
  
    int n = arr.size();  
    for (int i = 0; i < n - 1; i++)  
        for (int j = 0; j < n - i - 1; j++)  
            if (arr[j] > arr[j + 1])  
                swap(arr[j], arr[j + 1]);  
}
```

1.2 Insertion Sort

Insertion Sort builds the sorted array one element at a time by taking each element and inserting it into its correct position among the already-sorted elements to its left. It excels on nearly sorted or small datasets.

Time Complexity: $O(n^2)$ worst/average, $O(n)$ best. **Space Complexity:** $O(1)$.

Code:

```
void insertionSort(vector<int>& arr) {  
  
    int n = arr.size();  
    for (int i = 1; i < n; i++) {  
        int key = arr[i];  
        int j = i - 1;  
        while (j >= 0 && arr[j] > key) {  
            arr[j + 1] = arr[j];  
            j--;  
        }  
    }
```

```

        arr[j + 1] = key;
    }
}

```

1.3 Merge Sort

Merge Sort applies the divide-and-conquer strategy: the array is recursively split in half until each sub-array has one element, then sub-arrays are merged back in sorted order. It guarantees $O(n \log n)$ performance on all input patterns. **Time Complexity:** $O(n \log n)$ always. **Space Complexity:** $O(n)$.

Code:

```

void merge(vector<int>& arr, int l, int m, int r) {
    vector<int> left(arr.begin() + l, arr.begin() + m + 1);
    vector<int> right(arr.begin() + m + 1, arr.begin() + r + 1);
    int i = 0, j = 0, k = l;
    while (i < (int)left.size() && j < (int)right.size())
        arr[k++] = (left[i] <= right[j]) ? left[i++] : right[j++];
    while (i < (int)left.size()) arr[k++] = left[i++];
    while (j < (int)right.size()) arr[k++] = right[j++];
}

void mergeSort(vector<int>& arr, int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

void mergeSortWrapper(vector<int>& arr) {
    if (!arr.empty()) mergeSort(arr, 0, arr.size() - 1);
}

```

1.4 Quick Sort

Quick Sort selects a pivot, partitions the array so that elements less than the pivot are to its left and greater elements are to its right, then recursively sorts both partitions. The last-element pivot strategy used here degrades to $O(n^2)$ on sorted/reverse-sorted input. Randomized pivot selection would mitigate this. **Time Complexity:** $O(n \log n)$ average, $O(n^2)$ worst. **Space Complexity:** $O(\log n)$ stack.

Code:

```
int partition(vector<int>& arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
        if (arr[j] <= pivot) swap(arr[++i], arr[j]);
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSort(vector<int>& arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

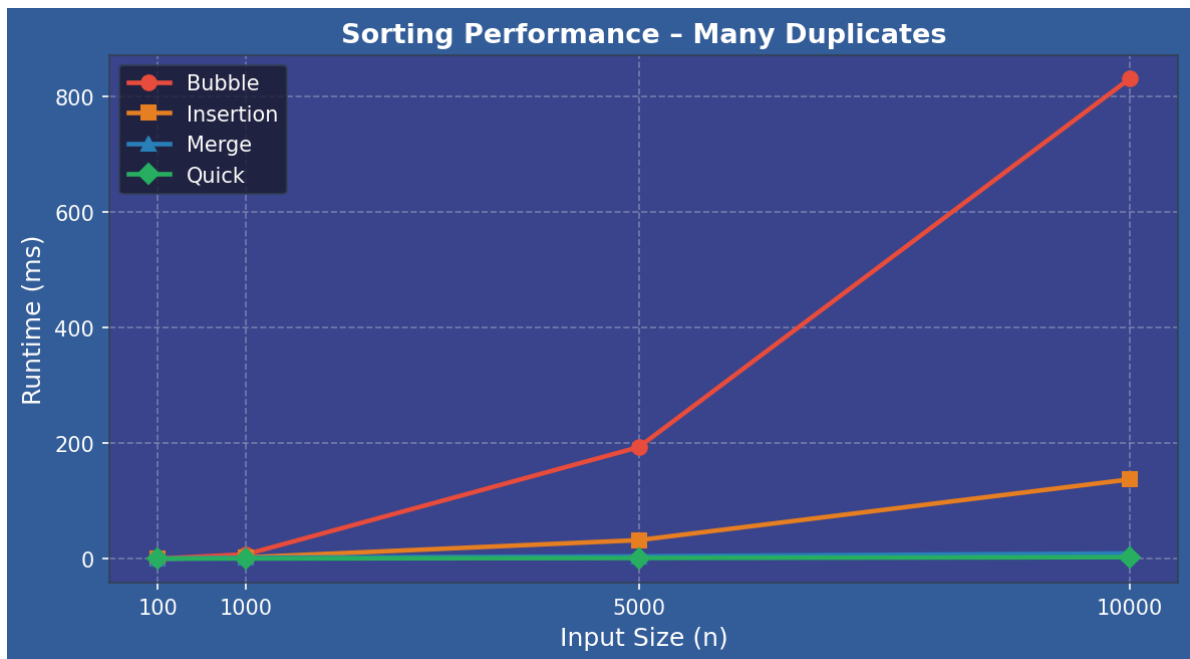
void quickSortWrapper(vector<int>& arr) {
    if (!arr.empty()) quickSort(arr, 0, arr.size() - 1);
}
```

2. Runtime Tables & Graphs

All measurements were taken using chrono: `high_resolution_clock` in c++17 compiled with `-O2`. Identical datasets were copied for each algorithm to ensure fairness.

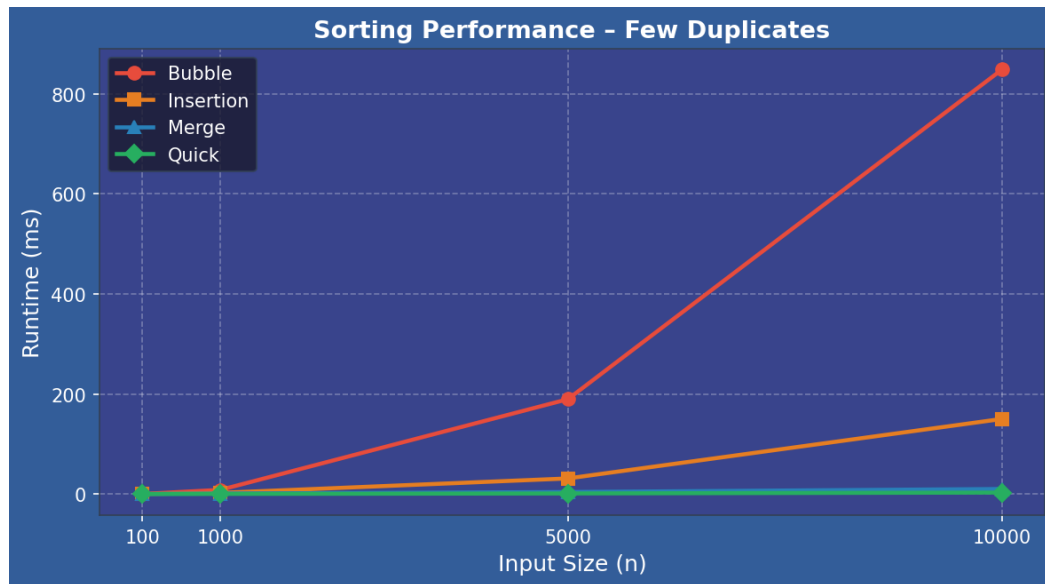
2.1 Many Duplicates

N	Bubble(ms)	Insertion(ms)	Merge(ms)	Quick(ms)
100	0.115	0.019	0.097	0.015
1000	7.590	2.325	1.585	0.331
5000	193.109	32.156	4.248	1.200
10000	830.659	137.279	9.143	2.665



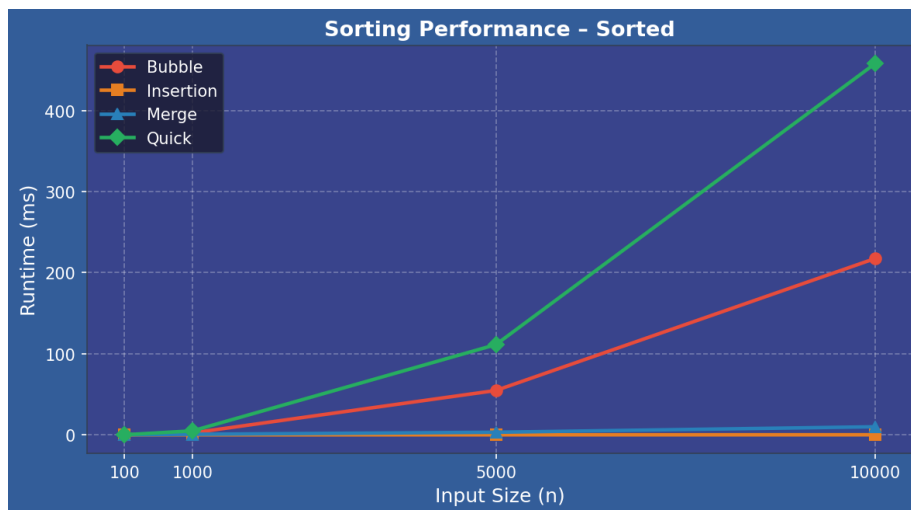
2.2 Few Duplicates

N	Bubble(ms)	Insertion(ms)	Merge(ms)	Quick(ms)
100	0.071	0.017	0.077	0.015
1000	8.332	2.307	0.883	0.205
5000	189.425	32.112	3.978	1.119
10000	848.207	149.994	10.082	2.904



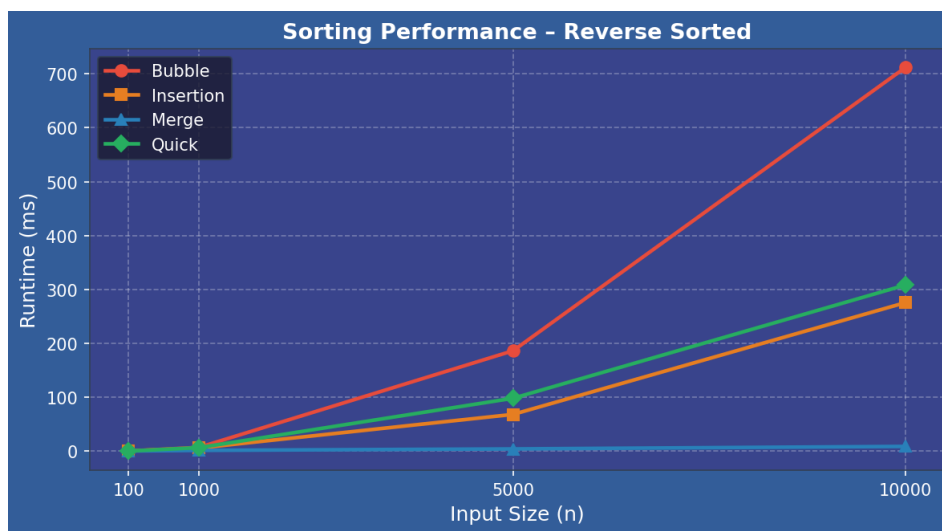
2.3 Sorted

N	Bubble(ms)	Insertion(ms)	Merge(ms)	Quick(ms)
100	0.030	0.002	0.084	0.057
1000	2.369	0.006	0.661	4.913
5000	54.677	0.029	3.174	111.221
10000	217.386	0.049	9.946	457.755



2.4 Reversed Sorted

N	Bubble(ms)	Insertion(ms)	Merge(ms)	Quick(ms)
100	0.077	0.027	0.080	0.031
1000	6.880	5.628	1.323	6.428
5000	186.096	68.053	4.010	97.987
10000	711.695	275.406	8.683	308.066



2.5 Discussions of Sorting Algorithm Behavior

As the input size increases, the gap between $O(n^2)$ and $O(n \log n)$ algorithms becomes very clear. Bubble Sort and Insertion Sort both slow down significantly as n grows, which is directly visible in the tables; their runtimes increase much faster compared to Merge Sort and Quick Sort. Between the two, Insertion Sort tends to be faster in practice because it stops shifting early once the correct position is found, whereas Bubble Sort always completes every pass.

Merge Sort stays consistent across all input patterns since it always divides and merges regardless of the data, making its runtimes stable across all four tables. Quick Sort is fast on random data but degrades on sorted and reverse-sorted inputs due to unbalanced partitions with a fixed pivot, which is visible when comparing those tables against the random ones.

The number of duplicates has a relatively small effect overall. Few duplicates mean more unique values and are closer to average-case behavior for all algorithms. Many duplicates can slightly affect how Quick Sort partitions the data, but do not change the general trend.

3. Stack, Queue, Circular Queue, Priority Queue

3.1 Stack(LIFO)

A stack follows Last-In-First-Out ordering. Elements are pushed onto and popped from the same end(top). It is implemented using a dynamic array (vector). All operations (push, pop, top, isEmpty, are $O(1)$).

Worked Example:

Operation	Stack State(top→bottom)	Output
Push(10)	[10]	Pushed 10
Push(20)	[20,10]	Pushed 20
Push(30)	[30,20,10]	Pushed 30
Display	[30,20,10]	
Peek()	[30,20,10]	30
Pop()	[20,10]	Popped 30
Pop()	[10]	Popped 20
Display	[10]	
isEmpty()	[10]	NO
Pop()	[]	Popped 10
isEmpty()	[]	Yes
Pop()	[]	Stack underflow

3.2 Queue (FIFO)

A queue follows First-In-First-Out(FIFO) ordering. Elements are enqueued at the rear and dequeued from the front. Implemented with a vector. All core operations are $O(1)$.

Operation	QUEUE	Output
Enqueue(10)	[10]	
Enqueue(20)	[10,20]	
Enqueue(30)	[10,20,30]	
Display()	[10,20,30]	[10,20,30]
Begin()	[10,20,30]	[10]
Back()	[10,20,30]	[30]
Size()	[10,20,30]	3
Dequeue()	[20,30]	[10]
Display()	[20,30]	[20,30]
isEmpty()	[20,30]	“NO”

3.3 Circular Queue

A circular queue uses a fixed-capacity array where front and rear indices wrap around using modular arithmetic($i = (i+1)\%capacity$). This eliminates the false-full problem of a linear queue. A worked example with CQ of size 5 is mentioned in the table below.

Operation	Front	Rear	Count	Queue Contents
Enqueue(1)	0	0	1	[1, , , ,]
Enqueue(2)	0	1	2	[1,2, , ,]
Enqueue(3)	0	2	3	[1,2,3, ,]
Enqueue(4)	0	3	4	[1,2,3,4,]
Enqueue(5)	0	4	5	[1,2,3,4,5]
Dequeue()	1	4	4	[,2,3,4,5]
Dequeue()	2	4	3	[, ,3,4,5]
Enqueue(6)	2	0	4	[6, ,3,4,5]
Enqueue(7)	2	1	5	[6,7,3,4,5] full

Priority Queue(Min Heap)

The **Priority Queue** is implemented as a binary min-heap stored in a vector. The smallest element is always at the root (index 0). `insert()` appends the element to the end and calls `heapify()` in upward mode, **$O(\log n)$** . `extractMin()` removes the root, replaces it with the last element, then calls `heapify()` in downward mode — **$O(\log n)$** . `peek()` returns the minimum in **$O(1)$** .

Operation	Heap Array	Heap Min(peek)
Insert(40)	[40]	40
Insert(10)	[10,40]	10
Insert(30)	[10,40,30]	10
Insert(5)	[5,10,30,40]	5
Insert(20)	[5,10,30,40,20]	5
<code>extractMin()</code> →5	[10,30,40,20]	10
<code>extractMin()</code> →10	[20,40,30]	20

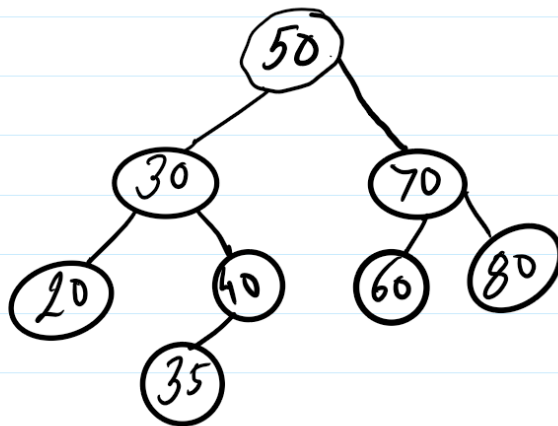
4. Binary Search Tree(BST)

A binary search tree maintains the invariant that for every node N: all keys in N's left subtree are less than N's key and all keys in N's right subtree are greater. This enables efficient $O(\log n)$ average search, insert, and delete operations. Worst case degrades to $O(n)$ on sorted inputs(produces a skewed tree).

4.1 Insertion

Starting at the root, compare the new key with the current node. The new element (if smaller) goes left and right (if larger).

BST After Insertions
50, 30, 70, 20, 40, 60, 80, 35



4.2 Traversals

Traversals	Order Produced	Use Case
Inorder(L-Root-R)	20 30 35 40 50 60 70 80	Sorted output
Preorder(Root-L-R)	50 30 20 40 35 70 60 80	Tree copying
Postorder(L-R-Root)	20 35 40 30 60 80 70 50	Tree deletion

4.3 Deletion- All Three Cases

Case	Node	Action	Result
Leaf node	20	Simply remove the node	30's left child becomes null
One Child	40	Replace the node with its single child	30's right child is replaced by it's one child
Two children	30	Find inorder successor(35)	30 replaced by 35; tree rebalanced

Inorder successor of a node is the smallest node in its right subtree. It always has at most one child (no left child), making its own deletion simple.

5. AVL Tree

An AVL tree is a self-balancing BST where the balance factor of every node (height of left subtree, height of right subtree) is maintained within $\{-1, 0, +1\}$. After every insertion, the tree is rebalanced via rotations if needed. Height can be calculated using $h(\text{node}) = 1 + \max(h(\text{left}), h(\text{right}))$. A null node has height 0. A leaf node has height 1. This guarantees $O(\log n)$ height and $O(\log n)$ operations.

5.1 Rotations and insertion examples for different conditions:

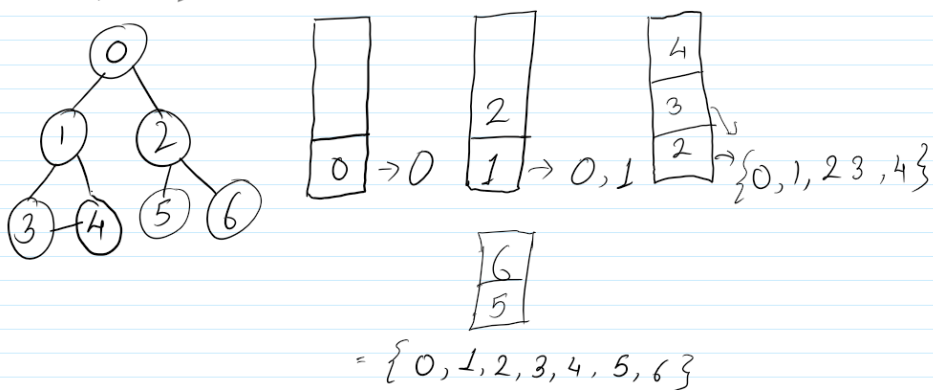
Rotation	Trigger Condition	Action
LL(Right Rotation)	BF > 1, left-left heavy	Single right rotation at unbalanced node
RR(Left Rotation)	BF < -1, right-right heavy	Single left rotation at unbalanced node
LR(Left-Right Rotation)	BF > 1, left-right heavy	Left rotate left child, the right rotate node
RL(Right-Left Rotation)	BF < -1, right-left heavy	Right rotate right child, then left rotate node

5. Graph Traversal: BFS & DFS

The graph is stored as an adjacency list using an unordered map. The example graph has $V=7$ nodes and $E=7$ edges, undirected.

Node	Neighbors
0	1,2
1	0,3,4
2	0,5,6
4	1,3
5	2
6	2

BFS (Queue)

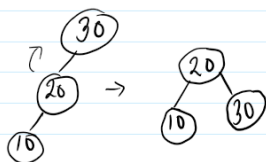


Time Complexity = $O(V+E)$

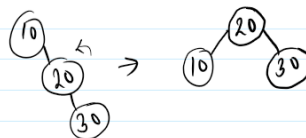
u

AVL

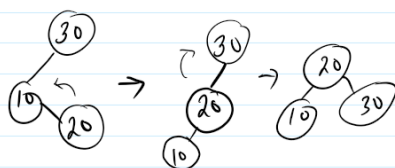
LL (30, 20, 10)



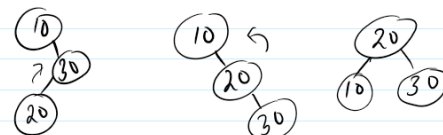
RR (10, 20, 30)



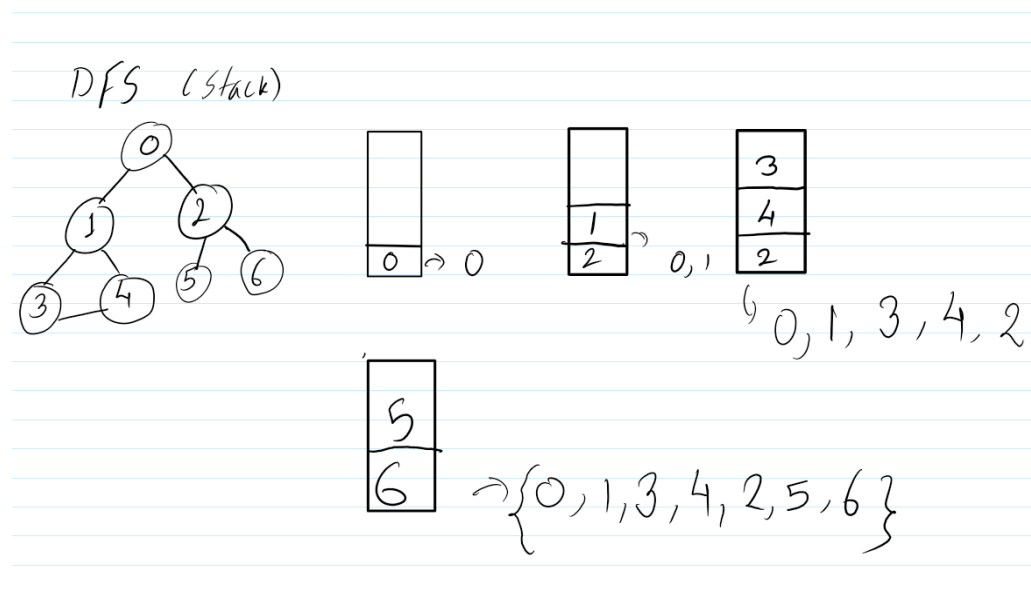
LR (30, 10, 20)



RL (10, 30, 20)



BFS uses a queue. Starting from node 0, it visits all nodes at distance 1 before nodes at 2, etc. The traversal order is; $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$. Time complexity: $O(V+E)$. Space: $O(V)$ for the visited set and queue.



DFS uses recursion (implicit call stack). It fully explores one branch before backtracking. The traversal order is: $0 \rightarrow 1 \rightarrow 3 \rightarrow 4 \rightarrow 2 \rightarrow 5 \rightarrow 6$. Time Complexity: $O(V+E)$. Space: $O(V)$ for the visited set and the recursion stack.

BFS is best for the shortest path(unweighted) and DFS is best for cycle detection, topological sort.

7. Theoretical Time Complexity Summary

Algorithm/Structure	Best Case	Average Case	Worst Case	Space
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Stack(push/pop)	$O(1)$	$O(1)$	$O(1)$	$O(n)$
Queue(enq/deq)	$O(1)$	$O(1)$	$O(1)$	$O(n)$
Circular Queue	$O(1)$	$O(1)$	$O(1)$	$O(n)$
Priority Queue	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$
BST Insert/Delete	$O(\log n)$	$O(\log n)$	$O(n)$	$O(n)$
BST Search	$O(\log n)$	$O(\log n)$	$O(n)$	$O(1)$
AVL Insert/Delete	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$
BFS	$O(V+E)$	$O(V+E)$	$O(V+E)$	$O(V)$
DFS	$O(V+E)$	$O(V+E)$	$O(V+E)$	$O(V)$

8. Conclusion

This project covered the full range of core topics from CSCE 3301 through hands-on implementation and experimental measurement. The sorting evaluation showed that algorithm selection depends heavily on both input size and input pattern, Bubble Sort and Insertion Sort are simple but scale poorly, while Merge Sort and Quick Sort handle large inputs efficiently under normal conditions, each with its own tradeoff between consistency and speed. The linear data structures demonstrated how Stack, Queue, Circular Queue, and Priority Queue each enforce a specific access order and deliver constant-time operations. The BST implementation showed how binary search tree structure enables efficient search and deletion, but also how a skewed tree degrades performance when input is sorted, a limitation that the AVL Tree directly addresses by maintaining balance through rotations after every insertion, guaranteeing logarithmic height at all times. Finally, BFS and DFS demonstrated two distinct ways to explore a graph, each suited to different problem types, both running in $O(V+E)$. Taken together,

the project reinforced that understanding theoretical complexity alongside actual measured behavior is essential for making good decisions about which algorithm or data structure to use in a given situation.